

SEMANA 2 — DÍA 2

Limpieza, Normalización y Validación con Expresiones Regulares

Objetivo del día: Aprender a limpiar datos crudos provenientes de una API, normalizarlos, validar campos con expresiones regulares y comenzar a construir el módulo `cleaner.py`.

TEORÍA DEL DÍA

Lee esta sección con calma. Todo lo que necesitas para trabajar está aquí.

1. ¿Qué es limpieza de datos?

La limpieza de datos es el proceso de transformar datos crudos en datos **útiles, consistentes y listos para análisis**.

En el PIA es necesaria porque:

- Las APIs devuelven **muchos campos irrelevantes**
- Algunos valores vienen como **strings cuando deberían ser números**
- Hay **estructuras anidadas** difíciles de usar
- Puede haber **valores nulos o faltantes**
- Los nombres pueden venir en formatos inconsistentes

Ejemplo típico de JSON crudo:

```
{
  "name": "pikachu",
  "height": "4",
  "weight": "60",
  "types": [
    {"type": {"name": "electric"}}
  ],
  "sprites": {...},
  "game_indices": [...]
}
```

2. Tipos de limpieza

2.1 Eliminar campos irrelevantes

```
if "sprites" in data:
    del data["sprites"]
```

2.2 Convertir tipos

```
altura = int(data["height"])
peso = float(data["weight"])
```

2.3 Normalizar nombres

```
nombre = data["name"].capitalize()
```

2.4 Aplanar estructuras anidadas

```
tipo = data["types"][0]["type"]["name"]
```

2.5 Manejar valores nulos

```
if data["felineIQ"] is None:  
    data["felineIQ"] = 0
```

3. Expresiones Regulares (Regex)

Las expresiones regulares permiten validar, buscar, dividir o reemplazar texto usando **patrones**.

Python usa el módulo:

`import re`

Para ver cada elemento a detalle, consulta:

 https://www.w3schools.com/python/python_regex.asp

4. Metacaracteres, Secuencias Especiales y Conjuntos

Las regex funcionan gracias a tres componentes clave:

4.1 Metacaracteres

Son símbolos con significado especial dentro de una expresión regular.

Metacarácter	Significado	Ejemplo
.	Cualquier carácter	a.c → "abc", "a-c"
^	Inicio de cadena	^Hola
\$	Final de cadena	mundo\$
*	0 o más repeticiones	a*
+	1 o más repeticiones	a+
?	0 o 1 repetición	colou?r
{}	Cantidad exacta	\d{4}
()	Agrupación	(abc)+

Ejemplo:

```
re.findall(r"colou?r", "color colour colouur")  
# ['color', 'colour']
```

4.2 Secuencias especiales

Combinaciones que representan clases de caracteres comunes.

Secuencia	Significado
\d	Dígito (0–9)
\D	No dígito
\w	Caracter alfanumérico
\W	No alfanumérico
\s	Espacio en blanco
\S	No espacio
\b	Límite de palabra

Ejemplo:

```
re.findall(r"\d+", "ID 123, código 456")  
# ['123', '456']
```

4.3 Conjuntos

Permiten definir rangos o grupos de caracteres válidos.

Conjunto	Significado
[abc]	a, b o c
[A-Z]	Letras mayúsculas
[a-z]	Letras minúsculas
[0-9]	Dígitos
[^0-9]	Cualquier cosa excepto dígitos

Ejemplo:

```
re.findall(r"[A-Za-z]+", "Hola123Mundo")  
# ['Hola', 'Mundo']
```

5. Funciones esenciales de regex en Python

5.1 re.findall() — Encuentra *todas* las coincidencias

```
texto = "ID: 123, Código: 456, Serie: 789"  
numeros = re.findall(r"\d+", texto)  
print(numeros) # ['123', '456', '789']
```

5.2 re.search() — Busca la *primera* coincidencia

```
texto = "Usuario: Ana"  
match = re.search(r"[A-Za-z]+", texto)  
  
if match:  
    print(match.group()) # Ana
```

5.3 re.split() — Divide un texto usando un patrón

```
texto = "manzana, uva; kiwi|pera"  
frutas = re.split(r"[;|]", texto)  
print(frutas)
```

5.4 re.sub() — Reemplaza coincidencias

```
texto = "  Hola  mundo  "  
limpio = re.sub(r"\s+", " ", texto).strip()  
print(limpio) # "Hola mundo"
```

6. Patrones típicos que pueden usar HOY en su PIA

Nombres válidos (solo letras)

```
r"^[A-Za-zÁÉÍÓÚáéíóúñÑ]+$"
```

IDs numéricos

```
r"^\d+$"
```

Correos electrónicos

```
r"^[\\w\\.\\-]+@[\\w\\.\\-]+\\.\\w+$"
```

Fechas (AAAA-MM-DD)

```
r"^\d{4}-\d{2}-\d{2}$"
```

EJEMPLOS PARA ESTUDIAR

Ejemplo 1: Limpiar un JSON crudo

```
import requests
import re

url = "https://swapi.dev/api/people/13"
response = requests.get(url)
data = response.json()
if data["species"] == []:
    especie = "humano"
else:
    response2 = requests.get(data["species"][0])
    data2 = response2.json()
    especie = data2["name"]

clean = {
    "nombre": data["name"].capitalize(),
    "altura": int(data["height"]),
    "peso": int(data["mass"]),
    "especie": especie
}
```

Ejemplo 2: Validar con regex

```
nombre = data["name"]

if re.search(r"^[A-Za-z]+$", nombre):
    nombre = nombre.capitalize()
else:
    nombre = "NombreInvalido"
```

Ejemplo 3: Aplanar estructuras

```
primer_tipo = data["types"][0]["type"]["name"]
```

Ejemplo 4: Eliminar campos irrelevantes

```
for campo in ["sprites", "game_indices", "moves"]:
    if campo in data:
        del data[campo]
```

Ejercicio de avance del PIA

Cada equipo debe:

1. Tomar su JSON crudo
2. Elegir **al menos 2 campos**
3. Aplicar limpieza:
 - eliminar ruido
 - convertir tipos
 - normalizar nombres
 - aplanar estructuras
4. Elegir **2 patrones regex** para validar
5. Crear un diccionario limpio
6. Imprimirlo en consola

TAREA

Crear archivo cleaner.py con:

- Funciones de limpieza
- Funciones de validación con regex
- Documentación básica
- Pruebas simples

Identificar qué campos de su API requieren:

- Conversión de tipos
- Normalización
- Aplanamiento
- Validación con regex